

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

"Сибирский государственный университет телекоммуникаций и
информатики"

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

ОТЧЕТ № 13
по дисциплине
"Web-технологии"

по теме:
ПОГРУЖЕНИЕ В РАЗРАБОТКУ НА JAVASCRIPT

Студент:
Группа ИКС-531

Д.А. Язиков

Преподаватель:
ст. преподаватель

А.В. Андреев

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....	6
1.1 JavaScript как язык программирования	6
1.2 Переменные и типы данных.....	6
1.3 Массивы и их методы	7
1.4 Управляющие конструкции.....	8
1.5 Функции	8
1.6 Объекты и работа с JSON.....	8
1.7 DOM API	9
1.8 Асинхронность: Promise и async/await	9
1.9 Подключение скриптов в HTML.....	10
2 ОПИСАНИЕ ПРОЕКТА.....	11
2.1 Технологический стек.....	11
2.2 Учебный API персонажей Marvel.....	11
2.3 Структура каталога lab13/	12
2.4 Архитектура сайта.....	13
2.5 Способ публикации	13
3 ХОД РАБОТЫ	15
3.1 Урок 1. Базовые понятия.....	15
3.1.1 Задача 1: имя и баланс пользователя	15
3.1.2 Задача 2: баланс через 7 дней	16
3.2 Урок 2. Основы JavaScript	16
3.2.1 Задача 1: вывод переписки в чате	16
3.2.2 Задача 2: поиск по тексту сообщений	17
3.3 Урок 3. Сайт <<Персонажи Marvel>>	18
3.3.1 Каркас страницы.....	18
3.3.2 Получение данных с API.....	19
3.3.3 Карточка персонажа	19
3.3.4 Модальное окно персонажа	20
3.3.5 Оркестрация: start.js	22

3.3.6	Локальная проверка	22
3.4	Урок 4. Публикация сайта	23
3.4.1	Автоматизация деплоя	23
ЗАКЛЮЧЕНИЕ		25
А ВОССТАНОВЛЕННЫЕ ИЛЛЮСТРАЦИИ		27

Примечание к архивной копии

Это архивная версия сохранившегося текста отчёта, а не повторно выполненная лабораторная работа. Исторические скриншоты отсутствуют, поэтому соответствующие рисунки в этой копии опущены. Иллюстрации в приложении получены повторным отображением сохранённых исходников и не заменяют оригинальных свидетельств выполнения. Сохранились JavaScript-файлы и код сайта Marvel. Показанные в приложении консольные результаты получены повторным запуском. Состояние старого API и прежнего сервера не проверено.

ВВЕДЕНИЕ

JavaScript --- один из трёх базовых языков веба (наряду с **HTML** и **CSS**). **HTML** описывает структуру документа, **CSS** задаёт его оформление, а **JavaScript** добавляет в страницу поведение: реакцию на действия пользователя, асинхронный обмен данными с серверами, изменение содержимого страницы без её перезагрузки. Без **JavaScript** современный веб остановился бы на статических документах --- любое интерактивное приложение (чат, веб-почта, интернет-магазин, картографический сервис) построено вокруг **JavaScript** в браузере.

Предыдущие лабораторные работы (№ 10, № 11, № 12) были посвящены проектированию интерфейса в графическом редакторе **Figma**: студент создавал статичный макет, но не оживлял его кодом. Текущая работа замыкает связку <<макет → код>>: на готовом **HTML**-каркасе с подключённым **Bootstrap 5** реализуется одностраничное приложение (<<*Single Page Application*>>), которое получает данные с публичного API через `fetch` и динамически собирает разметку карточек и модальных окон.

В качестве темы выбран сайт-каталог супергероев вселенной **Marvel**. Учебный API <https://jsfree-les-3-api.onrender.com/characters> возвращает JSON-массив из девятнадцати персонажей с полями `id`, `name` (русское имя), `nameor` (оригинальное имя), `description`, `thumbnail` (URL обложки) и `comics` (список комиксов с участием героя). Готовое приложение публикуется на собственном домашнем сервере под обратным прокси **Caddy** с автоматическим TLS-сертификатом Let's Encrypt и доступно по адресу <https://marvel.server34.netcraze.club>.

Цель работы --- получить практические навыки разработки на **JavaScript**: освоить базовые языковые конструкции, научиться работать с массивами, объектами и асинхронными запросами, реализовать пользовательский интерфейс на основе данных, полученных от внешнего API, и опубликовать готовое приложение в интернете.

Задачи:

1. Освоить базовые конструкции языка: переменные, типы данных, вывод в консоль (урок 1).

2. Изучить работу с массивами, циклами и строковыми методами (`includes`, фильтрация) на примере мини-мессенджера (урок 2).
3. Реализовать сайт-каталог героев Marvel: получить данные с учебного API через `fetch`, сформировать массив карточек и модальных окон, разместить их в DOM-дереве страницы (урок 3).
4. Развернуть приложение на публичном хостинге, обеспечить работу по защищённому протоколу HTTPS (урок 4).

Стенд:

- `Node.js 18+` --- среда исполнения JavaScript вне браузера, используется для запуска консольных задач уроков 1 и 2;
- современный браузер (Chrome, Firefox, Safari) --- среда исполнения JavaScript на стороне клиента, используется для запуска сайта;
- `Bootstrap 5.0.1 (jsDelivr CDN)` --- набор готовых CSS-компонентов и сетка, подключается одной строкой в `<head>`;
- учебный REST-API `jsfree-les-3-api.onrender.com/characters` --- источник данных о персонажах Marvel;
- локальный HTTP-сервер `python3 -m http.server` --- обход CORS-ограничений при открытии страницы по `file://`;
- собственный домашний сервер (Debian 12, Docker Compose) с обратным прокси Caddy и автоматическим выпуском TLS-сертификатов Let's Encrypt --- площадка для публикации готового сайта.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 JavaScript как язык программирования

JavaScript (часто сокращается до **JS**) --- интерпретируемый язык программирования с динамической типизацией, разработанный в *Netscape Communications* в 1995 году Бренданом Айком. Изначально создавался для добавления интерактивности в веб-страницы, в дальнейшем стандартизирован как **ECMAScript** (текущая редакция --- ES2024).

Ключевые особенности языка:

- **Интерпретируемость** --- код выполняется построчно средой исполнения (браузер, `Node.js`), без отдельного этапа компиляции в машинный код;
- **Динамическая типизация** --- тип переменной определяется значением во время выполнения и может меняться;
- **Прототипная объектная модель** --- наследование строится через цепочки прототипов, а не через классы (синтаксис `class` с ES2015 --- лишь синтаксический сахар над прототипами);
- **Функции первого класса** --- функции являются полноценными значениями: их можно сохранять в переменных, передавать в аргументы, возвращать из других функций;
- **Однопоточная асинхронная модель** --- ровно один поток исполнения и очередь задач (*event loop*); долгие операции (сетевые запросы, таймеры) выполняются вне основного потока и возвращают результат через колбэки или промисы.

1.2 Переменные и типы данных

Объявление переменных в современном JavaScript --- через ключевые слова `let` (изменяемая) и `const` (неизменяемая ссылка). Устаревшее `var` не используется: оно не поддерживает блочную область видимости и приводит к скрытым ошибкам.

```
const username = 'my name';    // строка, неизменяемая
let bonusBalance = 1000;       // число, изменяемая
```

```
const isActive = true;           // логический тип
const messages = [];             // массив объектов
const user = { name: 'Ann' };    // объект
let absent;                      // undefined
const empty = null;              // null
```

Listing 1.1 — Объявление переменных и базовые типы

Семь примитивных типов: `string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint`. Всё остальное (массивы, функции, даты, регулярные выражения, экземпляры классов) --- объекты.

1.3 Массивы и их методы

Массив в JavaScript --- индексируемая коллекция произвольных значений. Базовые методы:

- `push(x) / pop()` --- добавить/удалить элемент с конца;
- `forEach(fn)` --- вызвать функцию для каждого элемента;
- `map(fn)` --- получить новый массив, применив функцию к каждому элементу;
- `filter(fn)` --- получить новый массив только из тех элементов, для которых функция вернула `true`;
- `join(sep)` --- объединить элементы в строку через разделитель;
- `includes(value)` --- проверить, есть ли значение в массиве (`true/false`).

У строк есть одноимённый метод `includes`, который проверяет наличие подстроки:

```
Чёрный
' чай'.includesзелёный(''); // falseКажется
', пойдёмвкино '.includesкино(''); // true
```

Listing 1.2 — Поиск подстроки методом `includes`

1.4 Управляющие конструкции

Условный оператор `if/else if/else`, тернарный оператор `a ? b : c`, оператор выбора `switch`. Циклы: `for` (классический счётный), `for...of` (по значениям итерируемого объекта), `for...in` (по ключам объекта), `while` и `do while`.

```
for (let i = 0; i < messages.length; i++) {  
  const author = i % 2 === 0 ? Друг'' : Вы'';  
  console.log(author + ': ' + messages[i]);  
}
```

Listing 1.3 — Цикл с условным выбором имени отправителя

1.5 Функции

Функция в JavaScript --- объект, который можно вызывать. Способы объявления:

```
// 1. Декларация функции  
function sum(a, b) { return a + b; }  
  
// 2. Функциональное выражение  
const sum = function(a, b) { return a + b; };  
  
// 3. Стрелочная функция (ES2015)  
const sum = (a, b) => a + b;
```

Listing 1.4 — Три способа объявить функцию

Стрелочные функции компактны и не имеют собственного `this` --- удобно для колбэков. Декларации <<всплывают>> (*hoisting*) --- их можно вызывать выше по тексту, чем сама декларация. Функциональные выражения и стрелочные функции **не** всплывают.

1.6 Объекты и работа с JSON

Объект --- неупорядоченное множество пар <<ключ--значение>>. Запись объекта в текстовом виде --- **JSON** (*JavaScript Object Notation*) ---

де-факто стандарт обмена данными в вебе. Преобразование между JSON-строкой и объектом:

```
const obj = JSON.parse('{ "name": "Spider-Man", "id": 1009610 }');
const str = JSON.stringify(obj); // '{ "name": "Spider-Man", "id": 1009610 }'
```

Listing 1.5 — `JSON.parse` и `JSON.stringify`

1.7 DOM API

DOM (*Document Object Model*) --- объектная модель HTML-документа: страница представлена деревом узлов (`Element`, `Text`, `Comment`). Через JavaScript доступны методы поиска узлов и их изменения:

- `document.getElementById(id)` --- узел по атрибуту `id`;
- `document.querySelector(selector)` --- первый узел по CSS-селектору; `querySelectorAll` --- все совпадения;
- `element.innerHTML = '...'` --- заменить внутреннее содержимое узла на готовую HTML-строку (быстро, но требует самостоятельного экранирования пользовательских данных);
- `element.textContent = '...'` --- задать текст без интерпретации тегов (безопасно по умолчанию);
- `element.addEventListener(event, handler)` --- подписаться на событие (`click`, `submit`, `DOMContentLoaded` и др.).

Событие `DOMContentLoaded` срабатывает, когда браузер построил DOM-дерево (но ещё не загрузил картинки и стили). Это правильный момент для запуска скриптов, которые ищут элементы по `id`.

1.8 Асинхронность: **Promise** и **async/await**

Сетевые запросы выполняются в фоне и завершаются непредсказуемо -- JavaScript описывает такой результат объектом **Promise**. Промис находится в одном из трёх состояний: *pending* (ожидание), *fulfilled* (успех) или *rejected* (ошибка). Цепочки `.then(...)` / `.catch(...)` в современном коде заменены на синтаксис `async/await`, который делает асинхронный код визуально похожим на синхронный.

Запрос к серверу выполняется через встроенный `fetch`:

```
async function fetchCharacters() {
  const response = await fetch('https://jsfree-les-3-api.
    onrender.com/characters');
  if (!response.ok) {
    throw new Error('HTTP ' + response.status);
  }
  return await response.json();
}
```

Listing 1.6 — Асинхронный запрос с обработкой ошибки

`fetch` возвращает промис, который разрешается объектом `Response`; чтобы получить распарсенный JSON, нужен ещё один `await response.json()`. Ошибки сети попадают в `catch`, ошибки уровня HTTP (*4xx*, *5xx*) --- нет: `response.ok` проверяется вручную.

1.9 Подключение скриптов в HTML

JavaScript-файлы подключаются тегом `<script>` в `<head>` или в конце `<body>`. По умолчанию браузер блокирует разбор HTML на время загрузки и выполнения скрипта --- поэтому скрипты, которые работают с DOM, удобно размещать в конце `<body>`. Атрибуты `defer` и `async` позволяют загружать скрипты параллельно с разбором HTML; `defer` дополнительно гарантирует порядок выполнения и запуск после построения DOM. В учебном проекте выбран простой вариант --- три тега `<script>` в конце `<body>` (Bootstrap-бандл, `index.js`, `start.js`).

2 ОПИСАНИЕ ПРОЕКТА

2.1 Технологический стек

Все четыре урока решаются исключительно средствами браузера и `Node.js` --- без сборщиков, транспиляторов и менеджеров зависимостей. Это сознательное решение: цель курса --- показать, насколько мало <<ритуала>> требуется современному фронтенду.

- **HTML5** --- разметка единственной страницы каталога;
- **CSS3 + Bootstrap 5.0.1** --- готовая 12-колоночная сетка, утилитарные классы, компоненты `card`, `modal`, `navbar`, `spinner`;
- **Vanilla JavaScript (ES2022)** --- никаких фреймворков и сборки; синтаксис `async/await`, `template literals`, `Array.map`, опциональная цепочка `?.`; код выполняется браузером как есть;
- **Node.js 18+** --- запуск консольных задач уроков 1 и 2 без браузера.

Bootstrap подключён одной строкой в `<head>`, JS-бандл Bootstrap (для модальных окон) --- в конце `<body>`; никаких `npm install` и `node_modules`.

2.2 Учебный API персонажей Marvel

Источник данных --- учебный REST-API курса: <https://jsfree-les-3-api.onrender.com/characters>. Без авторизации, HTTPS, CORS открыт. По GET-запросу возвращает JSON-массив из девятнадцати героев. Структура одного объекта:

```
{
  "id": 1009610,
  "name": "Человекпаук",
  "nameor": "Spider-Man",
  "description": "Укушенный радиоактивным пауком ...",
  "modified": "2020-07-21T10:30:10-0400",
  "thumbnail": "http://i.annihil.us/u/prod/marvel/i/mg/3/50/526548a343e4b.jpg",
  "comics": [
```

```

        { "resourceURI": "...", "name": "A+X (2012) #4" },
        ...
    ],
    "urls": [ ... ]
}

```

Listing 2.1 — Фрагмент ответа API (один персонаж)

Полезные поля: `id` (уникальный идентификатор --- используется для связи карточки с модальным окном), `name` (русское имя), `nameor` (оригинальное имя), `description` (биография), `thumbnail` (URL обложки), `comics[]` (список комиксов). Поле `modified` и `urls` в проекте не используются.

Особенность. Изображения раздаются по протоколу `http`, а сайт публикуется на `https`. Браузер блокирует такой *mixed content*, поэтому в `index.js` URL картинки принудительно переписывается на `https` --- домен `i.annihil.us` поддерживает оба протокола.

API хостится на бесплатном тарифе платформы `Render`. При отсутствии запросов более 15 минут контейнер усыпляется и первый запрос может обрабатываться до 30 секунд (*cold start*). Это учтено в интерфейсе: на время загрузки в `#character-card-box` отображается спиннер `Bootstrap`, ошибка сети выводится понятным сообщением в `alert-danger`.

2.3 Структура каталога `lab13/`

Папка лабораторной работы разделена на четыре подкаталога по числу уроков:

```

lab13/ |—
  README.md |—Методичка
  (lab13).pdf |—
  les-1/ | |—
    task1.js      # пользователь + бонусный баланс | |—
    task2.js      # баланс через 7 дней |—
  les-2/ | |—
    task1.js      # вывод переписки | |—
    task2.js      # поиск по includes |—
  les-3/          # сайт Персонажи «Marvel» | |—
    index.html | |—
    index.js | |—

```

```

start.js └─
latex-report/          # этототчёт

```

Listing 2.2 — Файловая структура lab13/

2.4 Архитектура сайта

Сайт `les-3/` --- это *single-page application* (SPA), собирающий разметку прямо в браузере. Точка входа --- `index.html`: статический каркас на Bootstrap-компонентах, содержит навбар, заголовок, два пустых контейнера (`#character-card-box` и `#character-modal-box`) и спиннер-плейсхолдер. По событию `DOMContentLoaded` вызывается функция `start()` из `start.js`.

```

index.html | └─

    Bootstrap CSS (CDN, <head>) | └─
    Bootstrap JS bundle (CDN, конец <body>) | └─

    index.js ← fetchCharacters, getCharacterCard[s], |
                getCharacterModal[s], escapeHtml, toHttps | └─

    start.js ← start(): fetchCharacters → render

```

Listing 2.3 — Граф зависимостей файлов

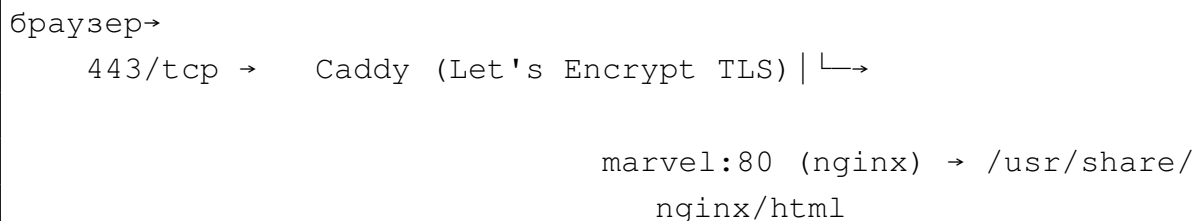
Разделение на `index.js` (чистые функции преобразования данных в HTML-строки) и `start.js` (оркестрация: запрос, рендер, обработка ошибок) повторяет учебный шаблон преподавателя и облегчает тестирование отдельных функций.

2.5 Способ публикации

Публикация сайта (урок 4 методички) выполнена не на коммерческом хостинге *Timeweb*, а на собственном домашнем сервере под управлением Debian 12. Сервер хостит несколько Docker-сервисов (Crafty, FileBrowser, FindMyDevice) под одним обратным прокси **Caddy**. Marvel-сайт добавляется в эту инфраструктуру как ещё один Docker-контейнер на образе

nginx:alpine, раздающий статические файлы `index.html`, `index.js`, `start.js`.

Логическая схема:



Listing 2.4 — Маршрут запроса до сайта

Адрес сайта --- <https://marvel.server34.netcraze.club>. TLS-сертификат выпускается автоматически Let's Encrypt при первом запросе по новому поддомену. DNS-запись `marvel` --- CNAME на основной домен `server34.netcraze.club`, который указывает на внешний IP домашнего роутера; роутер пробрасывает порты 80 и 443 на сервер.

Автоматизация деплоя --- идемпотентный `bash`-скрипт `home-server/scripts` синхронизирует файлы сайта в `/opt/stack/marvel/site/`, добавляет (если ещё нет) сервис `marvel` в общий `docker-compose.yml`, дописывает блок поддомена в `Caddyfile` и перезагружает `Caddy`. Подробное описание шагов скрипта --- в [главе 3](#).

3 ХОД РАБОТЫ

В этой главе по порядку описаны все четыре урока методички: консольные задачи на базовые конструкции JavaScript (уроки 1 и 2), реализация сайта-каталога (урок 3) и публикация сайта на собственном сервере с обратным прокси (урок 4). Для каждого урока приведён код решения, ожидаемый вывод и скриншоты результата.

3.1 Урок 1. Базовые понятия

Цель урока --- научиться объявлять переменные и выводить значения в консоль. Согласно методичке, моделируется бонусная программа интернет-магазина.

3.1.1 Задача 1: имя и баланс пользователя

Объявляются две переменные --- `username` и `bonusBalance`, --- значения которых выводятся в консоль двумя строками.

```
const username = 'my name';
const bonusBalance = 1000;

console.logПользователь(' ' + username);
console.logБаланс(' ' + bonusBalance);
```

Listing 3.1 — `les-1/task1.js` --- имя и баланс

Скрипт запускается командой `node task1.js` из каталога `les-1/`.
Вывод:

```
Пользователь
my nameБаланс
1000
```

Listing 3.2 — Вывод `task1.js`

3.1.2 Задача 2: баланс через 7 дней

За каждую покупку начисляется 50 бонусных баллов, ежедневно сгорает 3 балла. Покупки совершаются раз в два дня, первая --- во вторник. За 7 дней приходится 4 покупки (вт, чт, сб, пн), баланс изменяется на $4 \cdot 50 - 7 \cdot 3 = 200 - 21 = 179$ баллов.

```
let bonusBalance = 1000;
const purchaseBonus = 50;
const dailyBurn = 3;
const days = 7;

// Перваяпокупкавовторник    , дальшеразвдвдня    : вт, чт, сб,
    пн.
const purchases = 4;

bonusBalance = bonusBalance + purchases * purchaseBonus - days *
    dailyBurn;

console.logБаланс(' через ' + days + ' дней: ' + bonusBalance);
```

Listing 3.3 — les-1/task2.js --- баланс через 7 дней

Вывод:

```
Балансчерез
  7 дней: 1179
```

3.2 Урок 2. Основы JavaScript

Цель урока --- работа с массивами, циклами и строковыми методами на примере мини-мессенджера.

3.2.1 Задача 1: вывод переписки в чате

Инициализируется массив из четырёх сообщений; чётные индексы (0, 2, ...) принадлежат Другу, нечётные --- Вам. Используется цикл `for` с условным оператором, выбирающим имя отправителя.

```
const messages = [Пойдем
    ' гулятьвпарк  ?', Кажется
```

```

    ', дождь собирается . Лучше пойдём кино !', Давай
    ', сегодня как раз вышел новый фильм .', Встречаемся
    ' через час у кинотеатра .',
];

// Первое сообщение — от Друга , далее чередование .
for (let i = 0; i < messages.length; i++) {
    const author = i % 2 === 0 ? Друг : Вы;
    console.log(author + ': ' + messages[i]);
}

```

Listing 3.4 — les-2/task1.js --- вывод переписки

Вывод:

```

Друг
: Пойдём гулять в парк ? Вы
: Кажется, дождь собирается . Лучше пойдём кино ! Друг
: Давай, сегодня как раз вышел новый фильм . Вы
: Встречаемся через час у кинотеатра .

```

3.2.2 Задача 2: поиск по тексту сообщений

Реализован простейший поиск: тот же массив сообщений фильтруется методом `Array.prototype.filter` с проверкой `String.prototype.includes`. Найденные сообщения выводятся списком.

```

const messages = [Пойдём
    ' гулять в парк ?', Кажется
    ', дождь собирается . Лучше пойдём кино !', Давай
    ', сегодня как раз вышел новый фильм .', Встречаемся
    ' через час у кинотеатра .',
];

const searchText = 'кино';

console.log('Поиск по слову : "' + searchText + '"');
const found = messages.filter(msg => msg.includes(searchText));

if (found.length === 0) {
    console.log('Совпадений не найдено .');
} else {

```

```

    found.forEach(msg => console.log(' ' + msg));
}

```

Listing 3.5 — `les-2/task2.js` --- поиск через `includes`

Вывод:

```

Поиск по слову
: кино" — Кажется
, дождь собирается . Лучше пойдем в кино ! —
Встречаемся через час у кинотеатра
.

```

3.3 Урок 3. Сайт <<Персонажи Marvel>>

Главный артефакт лабораторной работы --- одностраничный сайт-каталог девятнадцати героев Marvel, реализованный в файлах `les-3/index.html`, `les-3/index.js`, `les-3/start.js`.

3.3.1 Каркас страницы

`index.html` --- статическая разметка на компонентах Bootstrap. Сразу после загрузки страница содержит тёмный навбар, заголовок <<Персонажи Marvel>> и спиннер-плейсхолдер; ниже --- два пустых контейнера для будущих карточек и модальных окон.

```

<div class="container">
  <h1 class="pt-3 pbПерсонажи-3"> Marvel</h1>

  <div class="row" id="character-card-box">
    <div class="d-flex justify-content-center">
      <div class="spinner-border text-danger" role="status">
        <span class="visually-hidden">Loading...</span>
      </div>
    </div>
  </div>

  <div id="character-modal-box"></div>
</div>

<script src="../../../bootstrap.bundle.min.js"></script>

```

```

<script src="index.js"></script>
<script src="start.js"></script>
<script>
    document.addEventListener("DOMContentLoaded", function() {
        start(); });
</script>

```

Listing 3.6 — Ключевая часть les-3/index.html

3.3.2 Получение данных с API

Функция `fetchCharacters` выполняет GET-запрос к учебному API и возвращает распарсенный JSON-массив. При ошибке HTTP выбрасывается исключение, которое перехватывается в `start()`.

```

const API_URL = 'https://jsfree-les-3-api.onrender.com/characters';

async function fetchCharacters() {
    const response = await fetch(API_URL);
    if (!response.ok) {
        throw new Error(`HTTP ${response.status}`);
    }
    return await response.json();
}

```

Listing 3.7 — les-3/index.js --- запрос к API

3.3.3 Карточка персонажа

`getCharacterCard` принимает один объект-персонаж и возвращает HTML-строку с Bootstrap-карточкой: обложка (`thumbnail`), имя по-русски, имя на оригинале, краткое описание (обрезается до 160 символов) и кнопка <<Подробнее>>, которая открывает модальное окно через атрибуты `data-bs-toggle` и `data-bs-target`.

```

function getCharacterCard(character) {
    const modalId = `character-modal-${character.id}`;
    const thumb = escapeHtml(toHttps(character.thumbnail));
    const name = escapeHtml(character.name);
    const nameor = escapeHtml(character.nameor);

```

```

    const short = escapeHtml(truncate(character.description, 160)
    );

    return `
<div class="col-12 col-md-6 col-lg-4 mb-4">
  <div class="card h-100 shadow-sm">
    
    <div class="card-body d-flex flex-column">
      <h5 class="card-title mb-1">${name}</h5>
      <p class="text-muted small mb-2">${nameor}</p>
      <p class="card-text flex-grow-1">${short}</p>
      <button type="button" class="btn btn-danger mt-2"
        data-bs-toggle="modal" data-bs-target="#${
          modalId}>Подробнее

      </button>
    </div>
  </div>
</div>`;
}

function getCharacterCards(characters) {
  return characters.map(getCharacterCard).join('');
}

```

Listing 3.8 — getCharacterCard --- HTML-карточка героя

Все значения, попадающие в HTML, пропускаются через `escapeHtml` --- защита от XSS на случай, если в данных встретятся служебные символы (<, >, &, кавычки). URL обложки переписывается с `http` на `https` (функция `toHttps`), иначе браузер заблокирует изображение из-за *mixed content* на HTTPS-странице.

3.3.4 Модальное окно персонажа

`getCharacterModal` собирает Bootstrap-модал с подробной биографией героя, увеличенной обложкой и списком до 10 комиксов с его участием.

Идентификатор модала формируется из id героя --- именно по нему карточка ищет нужный модал через data-bs-target.

```
function getCharacterModal(character) {
  const modalId = `character-modal-${character.id}`;
  const comics = Array.isArray(character.comics) ? character.
    comics.slice(0, 10) : [];
  const comicsList = comics.length
    ? `

${comics.map(c => `- ${escapeHtml(c.name)}</li>`
      >`).join('')}</ul>`
    : `

Список комиксов недоступен`</p>`;

  return `
<div class="modal fade" id="${modalId}" tabindex="-1">
  <div class="modal-dialog modal-lg modal-dialog-scrollable
    modal-dialog-centered">
    <div class="modal-content">
      <div class="modal-header"> ... </div>
      <div class="modal-body">
        <div class="row">
          <div class="col-md-4">  </div>
          <div class="col-md-8">
            <p>${escapeHtml(character.description)}</p>
            <h6>Появления в комиксах :</h6>
            ${comicsList}
          </div>
        </div>
      </div>
      <div class="modal-footer"> ... </div>
    </div>
  </div>
</div>`;
}

function getCharacterModals(characters) {
  return characters.map(getCharacterModal).join('');
}

```

Listing 3.9 — getCharacterModal --- модальное окно

(В листинге опущены повторяющиеся блоки `modal-header` и `modal-footer` --- полный код находится в репозитории.)

3.3.5 Оркестрация: `start.js`

Функция `start` вызывается по событию `DOMContentLoaded`: получает данные с API, заполняет оба контейнера готовой разметкой; при ошибке заменяет спиннер сообщением для пользователя.

```
async function start() {
  const cardBox = document.getElementById('character-card-box')
  ;
  const modalBox = document.getElementById('character-modal-box
    ');

  try {
    const characters = await fetchCharacters();
    cardBox.innerHTML = getCharacterCards(characters);
    modalBox.innerHTML = getCharacterModals(characters);
  } catch (err) {
    cardBox.innerHTML = `
<div class="col-12">
  <div class="alert alert-danger">Неудалось загрузить персонажей
    : ${err.message}.
  </div>
</div>`;
  }
}
```

Listing 3.10 — `les-3/start.js` --- запуск приложения

3.3.6 Локальная проверка

Открыть `index.html` напрямую через `file://` нельзя: браузер блокирует `fetch` к чужому домену. Поэтому для проверки поднимается локальный HTTP-сервер:

```
cd les-3 && python3 -m http.server 8000
```

Страница открывается по адресу `http://localhost:8000`. Через несколько секунд (или до 30 секунд при холодном старте API) спиннер сменяется сеткой карточек. Клик по кнопке <<Подробнее>> открывает модальное окно с биографией и списком комиксов.

3.4 Урок 4. Публикация сайта

Согласно методичке, готовый сайт должен быть выложен на публичный хостинг, а ссылка указана в отчёте. Вместо коммерческого Timeweb сайт развёрнут на собственном домашнем сервере, на котором уже работает обратный прокси **Caddy** с автоматическими TLS-сертификатами Let's Encrypt (см. главу 2).

3.4.1 Автоматизация деплоя

В корне `lab13/` лежит скрипт `deploy.sh` с двумя ключами `--install` и `--uninstall`. Оба режима идемпотентные, перезапуск ничего не ломает.

```
./deploy.sh --install      # развернуть сайт
./deploy.sh --uninstall    # снести сайт после показа
```

Listing 3.11 — Запуск деплоя и сноса сайта

Шаги режима `--install`:

1. Запрашивает у пользователя домен (по умолчанию `---marvel.server34` и путь к исходникам сайта (по умолчанию `---соседний каталог les-3/`).
2. Проверяет, что Caddy уже установлен (`50-install-caddy-proxy.sh` был запущен ранее); иначе `---` ошибка и выход.
3. Делает резервные копии `docker-compose.yml` и `Caddyfile` с меткой времени.
4. Через `rsync -a --delete` синхронизирует исходники `lab13/les-3/` в `/opt/stack/marvel/site/`.
5. Если в `docker-compose.yml` ещё нет сервиса `marvel` `---` добавляет: образ `nginx:alpine`, монтирует директорию сайта в `/usr/share/nginx/html` в режиме только для чтения.

6. Если в Caddyfile ещё нет блока для нашего поддомена --- дописывает в конец:

```
marvel.server34.netcraze.club {  
    encode gzip  
    reverse_proxy marvel:80  
}
```

7. Валидирует итоговый docker-compose.yml (docker compose config), запускает контейнер marvel и перезагружает Caddy (caddy reload).

Режим --uninstall выполняет обратные действия: останавливает и удаляет контейнер marvel, awk-фильтром вырезает сервис из docker-compose.yml и блок поддомена из Caddyfile, удаляет каталог /opt/stack/marvel/ и перезагружает Caddy. Бэкапы конфигов с меткой времени остаются рядом --- на случай отката.

Первый запрос к новому поддомену запускает выпуск TLS-сертификата Let's Encrypt; через несколько секунд страница открывается по HTTPS. Готовый адрес --- <https://marvel.server34.netcraze.club>.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы 13 получены практические навыки разработки на JavaScript, охватывающие путь от базовых конструкций языка до публикации полноценного веб-приложения.

Решены задачи всех четырёх уроков методички:

- на уроке 1 освоено объявление переменных (`const`, `let`), выбор типов данных (`string`, `number`) и вывод результатов в консоль через `console.log`;
- на уроке 2 проработана работа с массивами и циклами: итерация `for` с условным выбором имени отправителя, фильтрация массива методом `filter` в связке с `String.prototype.includes`;
- на уроке 3 реализован одностраничный сайт-каталог героев Marvel: данные получаются с публичного REST-API через `fetch/async-await`, разметка карточек и модальных окон формируется функциями `getCharacterCard[s]` и `getCharacterModal[s]` в связке с шаблонными строками; компоненты Bootstrap 5 (`card`, `modal`, `spinner`) обеспечивают внешний вид и адаптивность без единой строки собственного CSS;
- на уроке 4 готовое приложение опубликовано на собственном домашнем сервере под обратным прокси **Caddy** с автоматическим выпуском TLS-сертификата Let's Encrypt; адрес развёрнутого сайта --- <https://marvel.server34.netcraze.club>.

Дополнительные приёмы, не вынесенные в явные требования методички, но применённые в проекте:

- экранирование пользовательских данных через собственную функцию `escapeHtml` --- защита от XSS при использовании `innerHTML`;
- принудительный переход URL обложек с `http` на `https` --- обход блокировки *mixed content* на HTTPS-странице;

- обработка ошибки сети с информативным сообщением пользователю на месте спиннера;
- автоматизация развёртывания через идиоматичный `bash`-скрипт в репозитории `home-server`.

Полученные навыки --- основа любой клиентской разработки в вебе. Концепции событийной модели DOM, асинхронных запросов через промисы и шаблонных строк применимы как в проектах на чистом JavaScript, так и в большинстве современных фреймворков (*React*, *Vue*, *Svelte*), которые лишь дополняют изученный фундамент своими абстракциями (виртуальный DOM, реактивность, компоненты).

А ВОССТАНОВЛЕННЫЕ ИЛЛЮСТРАЦИИ

Все рисунки в этом приложении созданы при архивировании по сохранённым файлам проекта. Они не являются прежними скриншотами Figma, Codepen или удалённого сервера.

```
$ node les-1/task1.js

Пользователь my name
Баланс 1000
```

Рисунок 1 — Повторный запуск сохранённого JS: урок 1, задача 1

```
$ node les-1/task2.js

Баланс через 7 дней: 1179
```

Рисунок 2 — Повторный запуск сохранённого JS: урок 1, задача 2

```
$ node les-2/task1.js

Друг: Пойдем гулять в парк?
Вы: Кажется, дождь собирается. Лучше пойдем в кино!
Друг: Давай, сегодня как раз вышел новый фильм.
Вы: Встречаемся через час у кинотеатра.
```

Рисунок 3 — Повторный запуск сохранённого JS: урок 2, задача 1

```
$ node les-2/task2.js

Поиск по слову: "кино"
– Кажется, дождь собирается. Лучше пойдем в кино!
– Встречаемся через час у кинотеатра.
```

Рисунок 4 — Повторный запуск сохранённого JS: урок 2, задача 2